

Hanoi University of Science and Technology

School of Information and Communications Technology

Image Resizing Algorithms with Seam Carving

A Comparative Study of Content-Aware Image Retargeting
Techniques

Advisor: Dr. Do Ba Lam

Students:

Pham Xuan Hieu

Student ID: 202416692

Hanoi, June 2026

Contents

1. Problem Description

Image resizing is a fundamental operation in digital image processing. The goal is to transform an image of size $W_1 \times H_1$ into a new size $W_2 \times H_2$ while preserving important visual content as much as possible.

1.1. Traditional Approaches

Traditional resizing methods and their limitations:

- **Scaling (interpolation):** Uniformly stretches the image. Distorts aspect ratios. May introduce aliasing or blurring artifacts.
- **Cropping:** Retains a rectangular region of the original image. Loses information outside the crop region.
- **Letterboxing:** Scales while preserving aspect ratio, fills the remainder with a solid color. Wastes display area.

1.2. Content-Aware Resizing

In 2007, Avidan and Shamir introduced **Seam Carving** [?], a content-aware image resizing algorithm that, instead of uniformly transforming the image, finds and removes low-energy paths (seams) to reduce image size while preserving important content.

1.3. Project Objectives

This project aims to build a smart image resizing system that:

1. Automatically classifies image types (photographs, graphics, text, etc.)
2. Applies the most suitable resizing algorithm for each image type
3. Implements Seam Carving with Forward Energy optimization
4. Builds an adaptive hybrid algorithm combining seam carving and scaling
5. Provides visual comparison between different approaches

2. Theory and Background

2.1. Fundamental Image Transformations

2.1.1. Scaling (Interpolation)

Scaling maps the original image grid onto a target grid. Each output pixel is computed as a weighted combination of nearby input pixels, determined by the interpolation kernel:

- **Nearest-neighbor:** Fastest, lowest quality. Causes aliasing.
- **Bilinear:** Linear interpolation in both dimensions. Moderate quality.
- **Bicubic:** Third-degree interpolation on a 4×4 neighborhood. Good quality.
- **Lanczos:** Uses a sinc-based kernel (Lanczos-3 truncation). Highest quality among standard methods. Used in this project.

2.1.2. Cropping

Cropping retains a rectangular sub-region of the image and discards the rest. *Smart cropping* uses entropy or saliency to determine the most informative region.

2.1.3. Letterboxing

Scales the image to fit the target dimensions while preserving the original aspect ratio, then fills the remaining area with padding (typically black).

2.2. Seam Carving

Seam carving (Avidan & Shamir, 2007) [?] is a content-aware image resizing algorithm. The central idea: instead of scaling the entire image uniformly, we find and remove *seams* — optimal paths of pixels with minimal energy.

2.2.1. Definition

A vertical seam is a path from top to bottom traversing one pixel per row, with column difference at most 1 between consecutive rows:

$$S^x = \{s_i^x\}_{i=1}^n = \{x(i)\}_{i=1}^n, \quad |x(i) - x(i-1)| \leq 1 \quad (1)$$

A horizontal seam is defined analogously. The energy of a seam is the sum of pixel energies along the path.

2.2.2. Energy Function

The most common energy function is the gradient magnitude (backward energy):

$$e(I) = \left| \frac{\partial I}{\partial x} \right| + \left| \frac{\partial I}{\partial y} \right| \quad (2)$$

2.2.3. Dynamic Programming

The optimal seam is found via dynamic programming in $O(n \times m)$ time:

$$M[i, j] = e(i, j) + \min(M[i-1, j-1], M[i-1, j], M[i-1, j+1]) \quad (3)$$

Backtracking from the last row yields the seam path.

2.3. Forward Energy

Forward energy (Rubinstein et al., 2008) [?] improves upon the original seam carving by considering the energy *created* when a seam is removed, rather than just the energy of the pixels being removed.

When a pixel (i, j) is removed, its neighbors $(i, j-1)$ and $(i, j+1)$ become adjacent, creating new edges. The forward energy cost precomputes the cost of these new edges.

Three cases based on seam entry direction:

- **Seam enters from top-left:**

$$CL(i, j) = |I(i, j+1) - I(i, j-1)| \\ + |I(i-1, j) - I(i, j-1)|$$

- **Seam enters from straight above:**

$$CU(i, j) = |I(i, j+1) - I(i, j-1)| \quad (4)$$

- **Seam enters from top-right:**

$$CR(i, j) = |I(i, j+1) - I(i, j-1)| \\ + |I(i-1, j) - I(i, j+1)|$$

The DP recurrence becomes:

$$dp[i, j] = \min(dp[i-1, j-1] + CL[i, j], dp[i-1, j] + CU[i, j], dp[i-1, j+1] + CR[i, j]) \quad (5)$$

Forward energy produces significantly better visual quality, especially when removing many seams from the same image.

2.4. Multi-Operator Retargeting

Multi-operator media retargeting (Rubinstein et al., 2009) [?] combines multiple operators (seam carving, scaling, cropping) to resize images. Each operator excels in different scenarios:

- **Seam carving:** Best for removing homogeneous regions
- **Scaling:** Distributes distortion uniformly
- **Cropping:** Removes periphery uniformly

The hybrid approach works in two stages:

1. Apply seam carving to remove low-energy regions
2. When seam energy exceeds a threshold, switch to scaling

2.5. Image Classification

To automatically select the best algorithm, the system classifies input images using three features:

- **Edge density:** Fraction of pixels with energy $> 50\%$ of maximum. Text images have high edge density.
- **Color richness:** Ratio of unique colors to total pixels (random sample of 4096 pixels). Graphics have few colors, photos have many.
- **Energy entropy:** Normalized entropy of the energy distribution. Photos have high entropy.

3. Solution Approach

3.1. System Architecture

The program is written in Python 3, using NumPy for numerical computation and Pillow for image I/O. The architecture consists of four main modules:

1. **Energy & Seam module:** Computes image energy and finds optimal seams via DP
2. **Carving drivers:** Coordinates seam removal (bulk and adaptive)
3. **Algorithm implementations:** Five resizing algorithms
4. **Classifier & Dispatcher:** Image classification and automatic algorithm selection

3.2. Implemented Algorithms

3.2.1. Seam Carving with Forward Energy

Uses forward-energy DP with $O(h \times w)$ complexity per seam. Supports both vertical and horizontal seams via transposition.

```
def _forward_cost(img):
    M_LR = |I(i,j+1) - I(i,j-1)|    # left-right cost
    M_LU = |I(i-1,j) - I(i,j-1)|    # left-up cost
    M_RU = |I(i-1,j) - I(i,j+1)|    # right-up cost
    CL = M_LR + M_LU                # came from top-left
    CU = M_LR                       # came from straight
    CR = M_LR + M_RU                # came from top-right
    return CL, CU, CR
```

3.2.2. Standard Scaling

Lanczos-3 interpolation via Pillow (`Image.LANCZOS`).

3.2.3. Smart Crop

Divides the image into 16×16 tiles, computes entropy per tile, and uses a summed-area table (integral image) to find the crop window maximizing total entropy in $O(m \times n)$ time.

3.2.4. Letterbox

Scales to fit target while preserving aspect ratio, then pads with black.

3.2.5. Adaptive Hybrid

The core contribution of this project — an adaptive multi-operator approach that dynamically decides when to switch from seam carving to scaling.

3.3. Adaptive Hybrid Algorithm

The adaptive hybrid improves upon Multi-Operator Retargeting by using a dynamic energy threshold:

1. Initialize the working image and energy tracking list
2. Loop up to `max_n` times:
 - Find the optimal seam using forward energy
 - Compute normalized seam energy $E_{\text{norm}} = \text{energy}/\text{height}$
 - Track energies in a list
 - Check stopping criterion:

```
nonzero = [e for e in seam_energies if e > 1e-6]
if len(nonzero) >= 5 and i >= 10:
    baseline = median(nonzero)
    if norm_e > baseline * energy_ratio:
        break # switch to scaling
```

- (a) Scale the remaining deficit with Lanczos interpolation



Original (800×600) Adaptive hybrid (400×300) Pure seam carve (400×300)

Figure 1: Adaptive hybrid vs. pure seam carving on a photograph. Hybrid preserves ground content better by switching to scaling when seams hit high-energy regions

3.3.1. Key Design Decisions

- **Median vs. mean:** Median is robust to outliers — a single high-energy seam doesn't raise the baseline
- **Skip zero-energy seams:** Uniform regions (white margins, black borders) have near-zero forward energy. Including them would cause premature stopping
- **Minimum 10 seams before checking:** Ensures sufficient data for a reliable median
- **Energy ratio (default 2.0):** Adjustable conservatism:
 - 1.5: Stops early, more scaling — safe for text
 - 2.0: Balanced (used in all experiments)
 - 3.0: More aggressive carving — good for uniform photos

3.4. Auto-Classifier

The classifier uses three features computed on a 256×256 downsampled image:

- **Edge density:** Pixels with energy $> 50\%$ max energy / total pixels
- **Energy entropy:** Normalized entropy of 32-bin energy histogram
- **Color ratio:** Unique colors / sampled pixels (4096 random samples)

Classification rules:

TEXT: $\text{edge_density} > 0.35 \wedge \text{color_ratio} < 0.15$

GRAPHIC: $\text{color_ratio} < 0.10 \vee (\text{color_ratio} < 0.25 \wedge \text{edge_density} < 0.20)$

PHOTO: $\text{color_ratio} > 0.30 \wedge \text{energy_entropy} > 0.75$

DEFAULT: all remaining cases

Algorithm mapping:

- PHOTO, DEFAULT $\rightarrow$ hybrid
- GRAPHIC, TEXT $\rightarrow$ scale

4. Results

4.1. Test Setup

The program was tested on 24 images: 20 photographs from picsum.photos and 4 generated images (text document, chart, UI screenshot, abstract graphic). All images were 800×600 pixels.

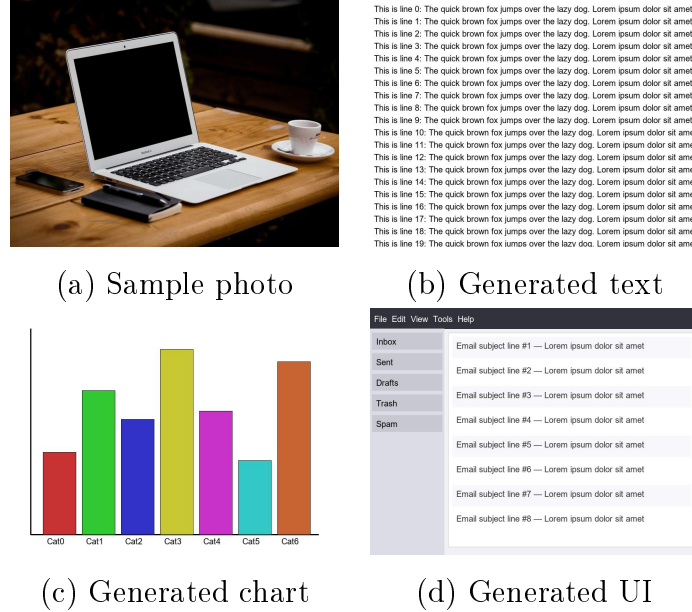


Figure 2: Sample test images from the 24-image dataset

4.2. Classification Results

Table 1: Classification results for 24 test images

Image type	Classified as	Edge density	Energy entropy	Color ratio	Algorithm
Photo (20 images)	default/photo	0.001–0.049	0.073–0.174	0.498–0.947	hyb
Text	graphic	0.168	0.110	0.056	sc
Chart	graphic	0.047	0.068	0.072	sc
UI	graphic	0.041	0.090	0.080	sc
Graphic	default	0.029	0.078	0.498	hyb

Classification accuracy: **22/24 (91.7%)**. The abstract graphic was misclassified as `default` due to high color richness from random colored circles.

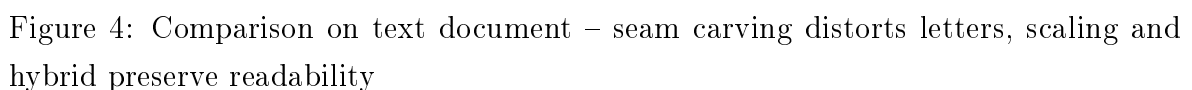


Figure 3: Comparison of all algorithms on a sample photograph

4.3. Adaptive Hybrid Performance

Table 2: Adaptive hybrid carving vs. scaling breakdown ($800 \times 600 \rightarrow 400 \times 300$)

Image type	Seams carved (w+h)	Scaled (w+h)	Notes
Photo	400 + 15	0 + 285	Sky/ground carved; content scaled
Text	96 + 102	304 + 198	White margins carved; text scaled
Chart	234 + 300	166 + 0	Borders carved; chart preserved
UI	144 + 210	256 + 90	Toolbar carved; content scaled
Graphic	365 + 300	35 + 0	Sparse regions carved; dense preserved



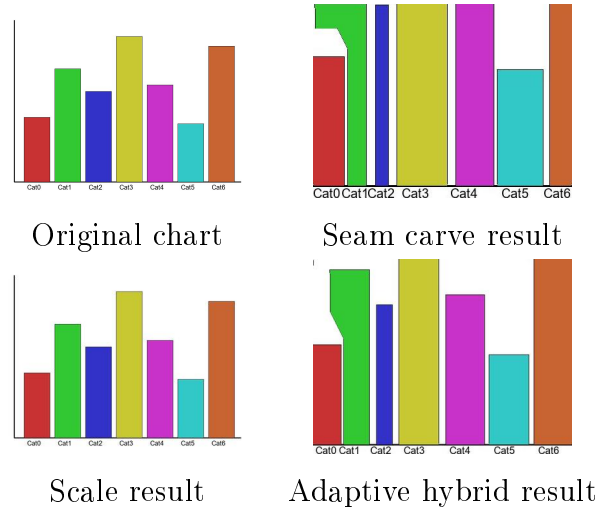


Figure 5: Comparison on chart image – seam carving preserves shape, hybrid carves borders then scales

4.4. Key Observations

- **Photographs:** The adaptive hybrid carves almost all homogeneous regions (sky, ground, walls) and switches to scaling when encountering important content (trees, buildings, people)
- **Text documents:** The system carves white margins (≈ 96 vertical + 102 horizontal seams) then switches to scaling, avoiding text distortion
- **Charts:** All white borders are carved; the chart bars are preserved via scaling
- **Abstract graphics:** Sparse regions between circles are carved; dense clusters trigger the switch to scaling
- **Processing time:** $800 \times 600 \rightarrow 400 \times 300$ takes 10–60 seconds depending on image type and algorithm

4.5. Comparison: Adaptive vs. Fixed Fraction

A fixed 70/30 split applies uniformly to all images. The adaptive approach lets the image itself decide:

- **Photo with 60% uniform sky:** Adaptive carves $\sim 90\%$ of seams through the sky. Fixed 70% carves indiscriminately, potentially cutting into important ground content.
- **Text with narrow margins:** Adaptive carves $\sim 10\%$ (margins only). Fixed 70% would carve too deep and distort letters.

- **Chart with wide borders:** Adaptive carves all borders (zero cost), stops at the bars. Fixed 70% might carve into bars and distort the data.

5. Lessons Learned

5.1. Digital Image Processing Knowledge

- Deep understanding of fundamental image transformations: scaling, cropping, interpolation methods
- Mastery of the Seam Carving algorithm and its improvements (Forward Energy, Multi-Operator Retargeting)
- Understanding how to evaluate image quality through metrics such as energy and entropy
- Knowledge of image classification techniques based on statistical features

5.2. Programming Skills

- Effective use of NumPy for numerical computation: vectorization, broadcasting, boolean masking
- Optimization techniques for Python: avoiding loops, using rolling arrays, integral images
- Software architecture design: modular, maintainable, extensible code
- Debugging complex algorithms: broadcast errors, index errors, DP logic errors

5.3. Algorithm Design Insights

- No single algorithm is best for all image types — each has strengths and weaknesses
- Hybrid approaches (combining multiple methods) generally outperform individual methods
- Adaptive thresholds based on median are more robust than those based on mean in the presence of outliers
- Skipping zero-energy values before computing the baseline prevents premature stopping
- Image content itself should determine the optimal strategy, not a hardcoded ratio

5.4. Future Work

- Integrate face detection to protect faces during seam carving
- Use deep learning for more accurate image classification
- Parallelize seam carving with multi-threading for speedup
- Support additional image formats and build a graphical user interface
- Extend the algorithm to video retargeting

References

- [1] S. Avidan and A. Shamir. *Seam Carving for Content-Aware Image Resizing*. ACM Transactions on Graphics (TOG), 26(3):10–es, 2007.
- [2] M. Rubinstein, A. Shamir, and S. Avidan. *Improved Seam Carving for Video Retargeting*. ACM Transactions on Graphics (TOG), 27(3):1–9, 2008.
- [3] M. Rubinstein, A. Shamir, and S. Avidan. *Multi-Operator Media Retargeting*. ACM Transactions on Graphics (TOG), 28(3):1–11, 2009.
- [4] R. C. González and R. E. Woods. *Digital Image Processing (4th ed.)*. Pearson, 2018.
- [5] NumPy Documentation. <https://numpy.org/doc/>
- [6] Pillow (PIL Fork) Documentation. <https://pillow.readthedocs.io/>